IFT 2035

IFT 2035 – Questions de pratique : évaluateurs et structures fonctionnelles

Questions **inventées** dans le style des intras (E2015, A2020, E2021, E2026) et du document de révision. Ce ne sont **pas** de vrais examens. Les types de Psil sont une **reconstruction** : ils peuvent différer un peu de ton `psil.hs`.

Méthode générale : donner les **types** d'abord, puis le **code**, puis le **coût**. Les fonctions doivent être **totales** (pas d'`error`, utiliser `Maybe`).

Partie 4 : Évaluateurs

Question 4.1 – Évaluateur (style intra E2021 §4)

⏱️ 20 minutes.

Soit l'évaluateur ci-dessous, écrit en Haskell, pour le langage Psil :

```
        type Var = ...                          type Env = ...
        data Value = Vnum Int                   data Lexp = Lnum Int
                   | Vobj Var Lexp                        | Lvar Var
                                                          | Lobjection Var Lexp     --
(objection () x e)
                                                          | Linvoke Lexp Lexp       -- (f e)
                                                          | Lbind Var Lexp Lexp     -- (bind
((x e1)) e2)
        env_lookup :: Env -> Var -> Value
        env_add    :: Env -> Var -> Value -> Env

        eval env (Lnum n) = n
        eval env (Lvar x) = env_lookup x env
        eval env (Lobjection x e) = Vobj x e
        eval env (Linvoke f actual)
          = case f of
              Vnum n -> error "Un nombre n'est pas une fonction"
              Vobj formal body ->
                eval (env_add env formal (eval actual)) body
```

Où `env_lookup` et `env_add` sont des fonctions qu'on présume prédéfinies, qui permettent respectivement de trouver la valeur d'une variable et d'ajouter une variable dans un environnement.

1. Haskell signale des erreurs de typage : indiquer comment corriger le code.
2. `eval` implémente-t-il l'appel par nom (CBN) ou l'appel par valeur (CBV) ? Justifier.
3. `eval` implémente-t-il la portée lexicale ou la portée dynamique ? Justifier.

4. Indiquer comment changer le code pour obtenir l'autre sorte de portée.
5. Haskell se plaint qu'`eval` n'est pas « exhaustive » : compléter sa définition. On veut que le `Lbind` soit **récursif** : la variable déclarée doit être visible dans sa propre définition.

Conseils :

- **1** : vérifie le type de chaque expression, ligne par ligne (il y a 4 erreurs). Regarde l'**ordre des arguments** de `env_lookup`, et ce que `case f of` analyse.
- **3** : où est la **photo** ? Est-ce que `Vobj` mémorise un environnement ?
- **5** : comment rendre un `let` récursif en Haskell ? Pense à la **paresse**.

Question 4.2 – Un évaluateur plein d'erreurs

```haskell
type Var = String

data Lexp = Lnum Int
          | Lvar Var
          | Linvoke Lexp Lexp                  -- (f e)
          | Lbind [(Var, Lexp)] Lexp           -- (bind ((x1 e1) ...) corps), DOIT
être récursif
          | Lobjection [(Var, Lexp)] Var Lexp  -- (objection ((a1 e1) ...) x corps)
          | Levidence Lexp Var                 -- (evidence o a)
          | Lif Lexp Lexp Lexp                 -- (if c alors sinon)

data Value = Vnum Int
           | Vobj [(Var, Value)] (Env -> Value -> Value)

type Env = [(Var, Value)]
```

```haskell
eval :: Env -> Lexp -> Value
eval env (Lnum n)               = n
eval env (Lvar x)               = lookup x env
eval env (Linvoke f a)          = case eval env f of
                                    Vobj _ fun -> fun env a
eval env (Levidence o a)        = case eval env o of
                                    Vobj attrs _ -> lookup a attrs
eval env (Lobjection as x body) =
  let attrs = [ (a, e) | (a, e) <- as ]
  in  Vobj attrs (\denv v -> eval ((x, v) : attrs ++ denv) body)
eval env (Lbind decls body)     =
  let env1 = [ (x, eval env e) | (x, e) <- decls ] ++ env
  in  eval env1 body
eval env (Lif c t e)            = case eval env c of
                                    Vnum 0 -> eval env t
                                    _      -> eval env e
```

`lookup :: Eq a => a -> [(a, b)] -> Maybe b` est prédéfinie.

1. **Erreurs de type.** Trouver toutes les erreurs de type (il y en a 5). Pour chacune : la ligne, le type attendu, le type obtenu, et une correction qui ne change pas ce que le code essaie de faire.

2. **Erreur dans `Lif`.** Une fois les types corrigés, que donne `(if 0 1 2)` ? Est-ce le comportement attendu, si `0` signifie « faux » ? Corriger.
3. **Portée.** Une fois les types corrigés, cet évaluateur a-t-il la portée lexicale ou dynamique ? Quelles lignes le montrent ? Le **type `Value`** lui-même le trahit : expliquer comment.
4. **Corriger la portée.** Modifier le type `Value` et les lignes nécessaires pour obtenir la portée lexicale.
5. **Récursivité.** Le `Lbind` doit être récursif. L'est-il ? Que donne `(bind ((f (objection () n (f n)))) …)` avec ce code ? Corriger la ligne 12, et expliquer pourquoi la version corrigée ne boucle pas en Haskell.
6. **Prédire**, avec ta version entièrement corrigée (lexicale, `Lbind` récursif), puis avec la version dynamique :

   ```
   (bind ((n 1000)
          (mk (objection () k (objection () x (+ x k)))))
     (bind ((k 1)) ((mk 5) 2)))
   ```

7. **Valeur ou nom.** Avec ta version corrigée, que donne `((objection () x 1) (/ 1 0))` ? Est-ce un appel par valeur ? Que faut-il ajouter à la ligne 5 pour en faire un vrai ?

Conseils :

- **1** : vérifie le type de chaque côté de chaque `=`, et de chaque argument passé à une fonction. Les endroits suspects : `lookup` (qui rend un `Maybe`), et tout ce qui reçoit une `Lexp` au lieu d'une `Value`.
- **3** : d'où vient l'environnement utilisé pour évaluer `body` ? De la création de l'objet, ou de l'appel ?

Question 4.3 – Évaluateur Psil à compléter

```haskell
        type Var = String

        data Lexp = Lnum Int                          -- 42
                  | Lvar Var                          -- x
                  | Linvoke Lexp Lexp                 -- (f e) : appel avec un argument
                  | Lbind [(Var, Lexp)] Lexp          -- (bind ((x1 e1) ... (xn en)) corps) :
RÉCURSIF
                  | Lobjection [(Var, Lexp)] Var Lexp
                       -- (objection ((a1 e1) ... (an en)) x corps) :
                       -- une fonction de paramètre x, avec des attributs a1 ... an
                  | Levidence Lexp Var                -- (evidence o a) : lit l'attribut a de
l'objet o
                  | Lif Lexp Lexp Lexp

        data Value = Vnum Int
                   | Vbool Bool
                   | Vobj [(Var, Value)] (Value -> Value)
                       -- un objet : ses attributs, et la fonction à appeler

        type Env = [(Var, Value)]
```

```haskell
eval :: Env -> Lexp -> Value
eval env (Lnum n)               = Vnum n
eval env (Lvar x)               = case lookup x env of
                                    Just v  -> v
                                    Nothing -> error ("variable inconnue : " ++ x)
eval env (Linvoke f a)          = case eval env f of
                                    Vobj _ fun -> ???
                                    _          -> error "pas une fonction"
eval env (Levidence o a)        = ???
eval env (Lobjection as x body) =
  let attrs = [ (a, eval env e) | (a, e) <- as ]
  in  Vobj attrs (\v -> eval ((x, v) : attrs ++ env) body)
eval env (Lbind decls body)     =
  let env1 = [ (x, eval env1 e) | (x, e) <- decls ] ++ env
  in  eval env1 body
eval env (Lif c t e)            = ???
```

1. Écrire la `Lexp` qui correspond à `(bind ((x 1) (o (objection ((x 10)) y (+ x y)))) (o 5))`. On suppose que `+` est une variable de l'environnement initial, liée à un objet sans attribut.
2. Compléter `Linvoke`, `Levidence` et `Lif`. Pour `Lif`, tout ce qui n'est pas `Vnum 0` ni `Vbool False` est vrai.
3. Dans l'équation `Lobjection` :
   - (a) dans quel environnement sont évalués les attributs ?
   - (b) un attribut peut-il voir le paramètre `x` ? Un autre attribut ?
   - (c) le corps peut-il voir les attributs ?
   - (d) si un attribut et le paramètre ont le même nom, lequel gagne ? Quelle partie du code le décide ?
4. Prédire la valeur de :
   - (a) `(bind ((x 1) (o (objection ((x 10)) y (+ x y)))) (o 5))`
   - (b) `(bind ((o (objection ((x 10)) x x))) (o 5))`
   - (c) `(bind ((a 100)) ((objection ((a 1) (b a)) x b) 0))`
5. **Récursivité.** Dans l'équation `Lbind`, la définition de `env1` utilise `env1` elle-même. Pourquoi est-ce que ça ne boucle pas en Haskell ? Que se passerait-il dans un langage strict ?
6. **Portée.** Cet évaluateur a-t-il la portée lexicale ou dynamique ? Quelle ligne le décide ? Que faudrait-il changer (dans `Value`, `Lobjection` et `Linvoke`) pour avoir la portée dynamique ?
7. **Valeur ou nom.** Prédire `((objection () x 1) (/ 1 0))`. Le résultat correspond-il à l'appel par valeur ? Modifier `Linvoke` pour forcer un vrai appel par valeur (indice : `seq`).

Partie 5 : Structures de données fonctionnelles

Question 5.1 – File à deux listes

```haskell
data Queue a = Queue [a] [a]
```

- la première liste contient les éléments **à sortir**, dans l'ordre de sortie ;
- la seconde liste contient les éléments **ajoutés récemment**, dans l'ordre **inverse** de leur arrivée.

Par exemple, la file qui contient 1, 2, 3, 4 (1 sortira en premier) peut s'écrire `Queue [1, 2] [4, 3]`.

```haskell
emptyQ = Queue [] []
```

1. Donner le type de `emptyQ`.
2. Donner des types raisonnables pour les fonctions suivantes (totales) :
   - (a) `enqueue` : ajoute un élément à la fin de la file ;
   - (b) `dequeue` : enlève le premier élément de la file, et le renvoie avec la nouvelle file ;
   - (c) `isEmpty` : indique si la file est vide.
3. Écrire le code de `enqueue` et `dequeue`. *Indice : que faire si la première liste est vide, mais pas la seconde ?*
4. À partir de `emptyQ`, on fait : `enqueue 1`, `enqueue 2`, `enqueue 3`, puis `dequeue`, puis `enqueue 4`, puis `dequeue`. Écrire la valeur de la file (`Queue … …`) après chaque opération.
5. Indiquer l'ordre de grandeur du nombre d'opérations en temps CPU et en allocations mémoire pour `enqueue` et pour `dequeue` :
   - (a) dans le pire cas ;
   - (b) en moyenne sur une longue suite d'opérations.
6. *(Bonus)* Pourquoi n'utilise-t-on pas une seule liste, en ajoutant à la fin avec `xs ++ [x]` ?

Question 5.2 – Zipper

```haskell
data Zipper a = Zipper [a] a [a]
```

- la première liste contient les éléments **à gauche** du curseur, **à l'envers** (le plus proche en premier) ;
- le champ du milieu est l'élément **sous** le curseur ;
- la dernière liste contient les éléments **à droite** du curseur, dans l'ordre.

Par exemple, la liste `[1, 2, 3, 4, 5]` avec le curseur sur `3` s'écrit `Zipper [2, 1] 3 [4, 5]`.

1. Donner des types raisonnables pour les fonctions suivantes (totales) :
   - (a) `fromList` : crée un zipper à partir d'une liste, curseur sur le premier élément ;
   - (b) `left` / `right` : déplacent le curseur d'une position ;
   - (c) `insert` : insère un élément à gauche du curseur ;
   - (d) `toList` : rend la liste complète, dans l'ordre.
2. Écrire `fromList`, `right` et `toList`.
3. Que doit faire `right` quand le curseur est déjà tout à droite ?
4. Coût en temps et en allocations de `right`, `insert` et `toList`, pour une liste de N éléments.

Question 5.3 – S-expressions

```haskell
data Sexp = Snil              -- la liste vide ()
          | Ssym String       -- un symbole, par ex. foo
          | Snum Int          -- un nombre, par ex. 42
          | Scons Sexp Sexp   -- une paire : (tête . queue)
```

Une **liste** Lisp est une suite de `Scons` qui se termine par `Snil`. Par exemple, `(+ 1 2)` s'écrit :

```
        Scons (Ssym "+") (Scons (Snum 1) (Scons (Snum 2) Snil))
```

(On utilise ici l'ordre habituel de Lisp, tête puis queue. Ce n'est peut-être pas l'ordre de ton TP1.)

1. Écrire la valeur `Sexp` qui représente : (a) `(f x)` ; (b) `(a (b c) d)` ; (c) `((1 2) 3)`.
2. Donner des types raisonnables pour les fonctions suivantes (totales) :
   - (a) `sexpLength` : la longueur d'une liste Lisp ;
   - (b) `sexpToList` : convertit une liste Lisp en liste Haskell de `Sexp` ;
   - (c) `sexpNth` : renvoie le n-ième élément d'une liste Lisp ;
   - (d) `countSyms` : compte tous les symboles d'une `Sexp`, à toutes les profondeurs.
3. Une `Sexp` n'est pas toujours une liste bien formée. Donner un exemple de `Sexp` qui n'est pas une liste, et dire ce que doit renvoyer `sexpToList` dans ce cas.
4. Écrire le code de `sexpToList` et de `countSyms`. Vérifier `countSyms` sur `(a (b c) d)` : le résultat doit être 4.
5. Indiquer l'ordre de grandeur en temps CPU et en allocations mémoire pour :
   - (a) `sexpNth n` sur une liste de longueur N ;
   - (b) `countSyms` sur une `Sexp` contenant N `Scons` au total ;
   - (c) ajouter un élément au début d'une liste Lisp, puis à la fin.

Question 5.4 – S-expressions et arbre binaire de recherche

```
        data TreeMap b = Empty | Node Int b (TreeMap b) (TreeMap b)

        data Sexp = Snil | Ssym String | Snum Int | Scons Sexp Sexp
```

On veut **sauvegarder** un arbre dans un fichier, sous forme de S-expression, puis le **relire**. Encodage :

- l'arbre vide `Empty` est représenté par `()`, c'est-à-dire `Snil` ;
- un nœud `Node k v g d` est représenté par la liste de 4 éléments `(k v g' d')`, où `g'` et `d'` sont les encodages des sous-arbres.

Par exemple, `Node 5 (Ssym "x") Empty Empty` est encodé par `(5 x () ())`.

1. Écrire, en notation Lisp, l'encodage de :

   ```
           t = Node 50 (Ssym "e") (Node 30 (Ssym "c") Empty Empty)
                                  (Node 70 (Ssym "g") Empty Empty)
   ```

2. Donner des types raisonnables pour : (a) `toSexp`, qui encode un `TreeMap Sexp` en `Sexp` ; (b) `fromSexp`, qui décode une `Sexp` en `TreeMap Sexp`. Expliquer pourquoi les deux types ne sont pas symétriques.
3. Donner deux exemples de `Sexp` que `fromSexp` doit refuser, pour deux raisons différentes.
4. Écrire `toSexp`. *Indice : écris d'abord `list4 :: Sexp -> Sexp -> Sexp -> Sexp -> Sexp`.*
5. Écrire `fromSexp`.
6. Écrire `sexpLookup :: Int -> Sexp -> Maybe Sexp`, qui cherche une clé directement dans l'encodage, sans le décoder d'abord.
7. Coût en temps et en allocations de : (a) `toSexp` pour un arbre de N nœuds ; (b) `sexpLookup` comparé à `tmLookup`, pour un arbre de profondeur h. Lequel est le plus rapide, et pourquoi la différence reste-t-elle petite ?

Conseil (5 et 6) : le motif qui reconnaît exactement une liste de 4 éléments est `Scons (Snum k) (Scons v (Scons g (Scons d Snil)))`.

Checklist avant de rendre

```
□ Types d'abord, code ensuite
□ Fonctions totales : Maybe au lieu de error
□ Un cas par constructeur ; toutes les combinaisons de motifs couvertes
□ Recopier le chemin, partager le reste (coût en allocations)
□ Si l'énoncé donne un exemple, je déroule mon code dessus
□ Évaluateur : dans quel environnement est évalué le corps ? (photo = lexical, appel = dynamique)
□ Évaluateur : l'argument est-il calculé avant l'appel ? (Haskell est paresseux : seq)
```